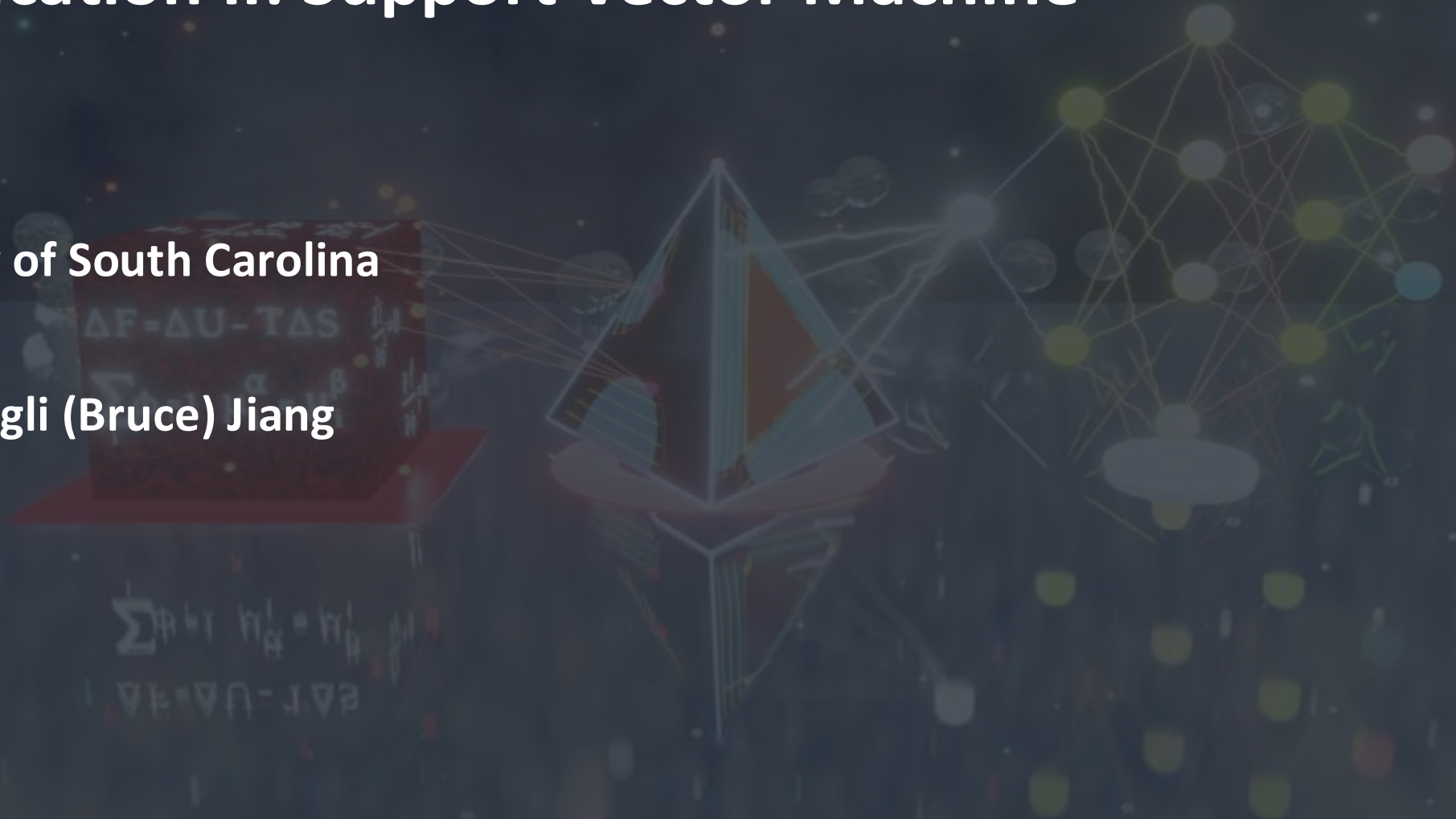


Classification II: Support Vector Machine

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 12

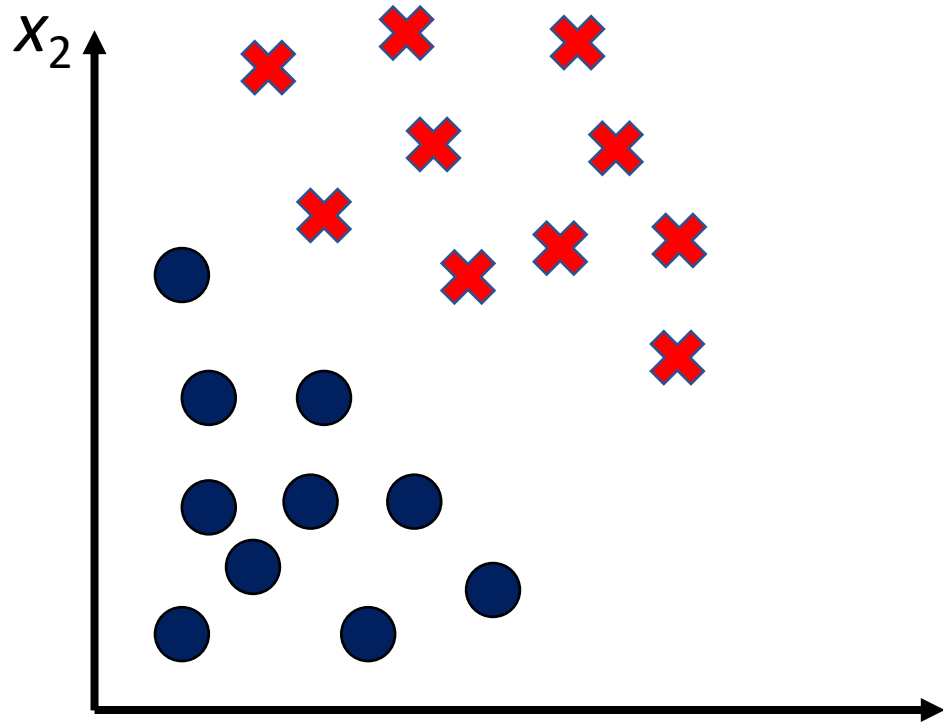
Classification: Logistic Regression

- *Logistic regression converts a linear score into a class probability.*
- *Classification metrics evaluate predicted classes.*
- *Multiclass problems can use one-vs-rest or one-vs-one.*

In this lecture, you will learn

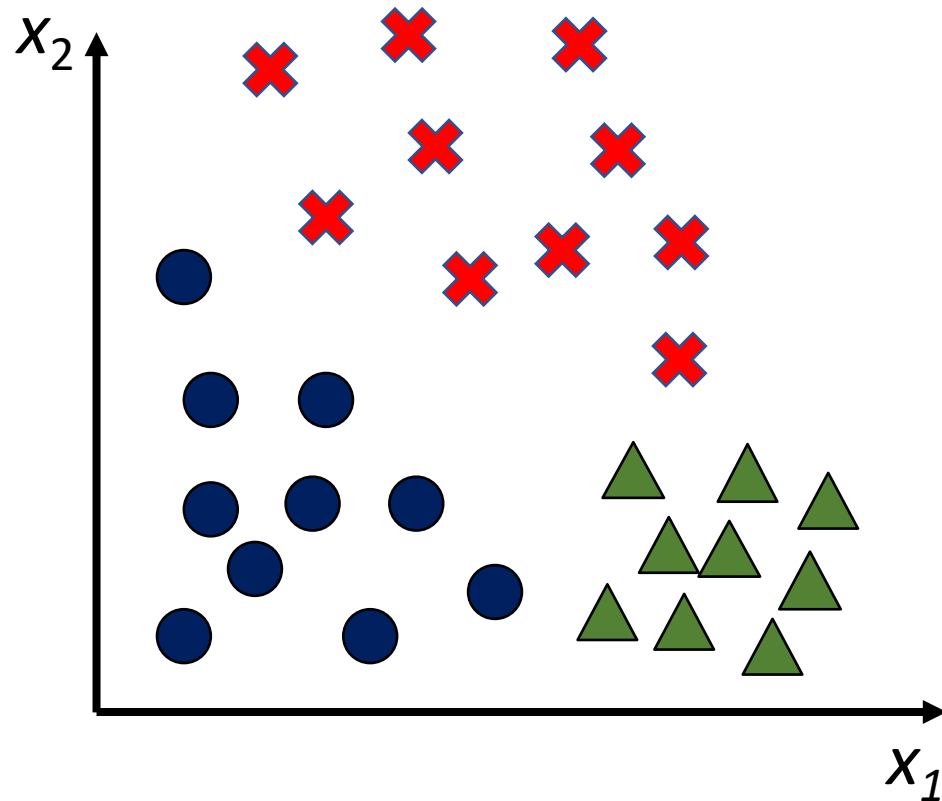
- *Explain the SVM decision boundary, margin, and support vectors.*
- *Explain soft-margin SVM and the role of C .*
- *Train and evaluate a linear SVM.*

Application now to classification



- **Binary case:** we just need to find the optimal decision boundary that partitions these classes

Application now to classification: One vs. All (Rest)



- **Binary case:** we just need to find the optimal decision boundary that partitions these classes

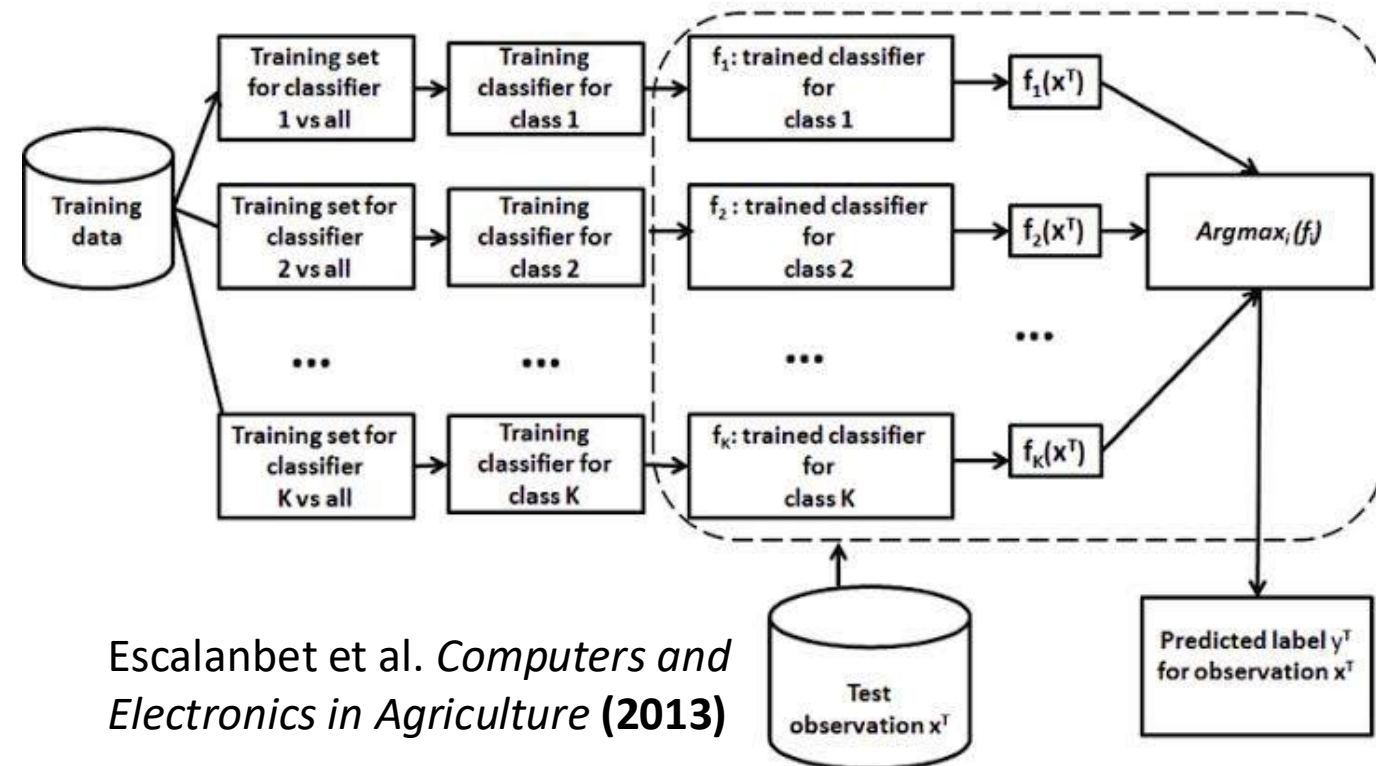
what do we do for multiple classes?

- **Multiclass case:** there are multiple strategies

Consider our data to have N classes...

One vs. All (Rest)

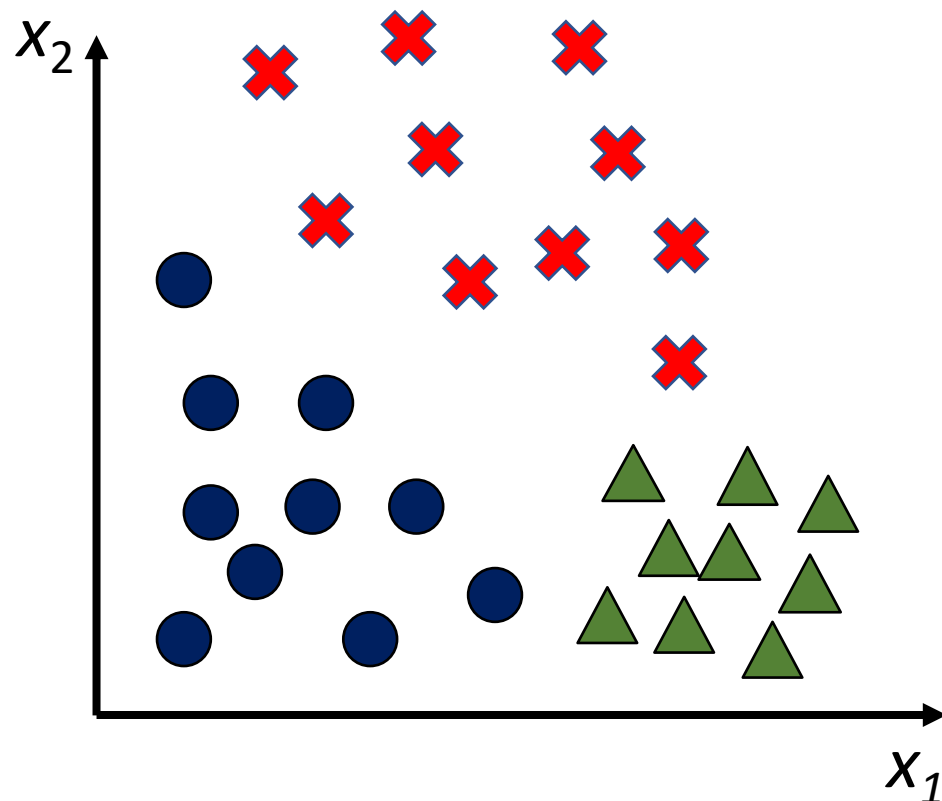
- formulate N binary classifier models



Escalanbet et al. *Computers and Electronics in Agriculture* (2013)

- pick the class that exhibits the highest score

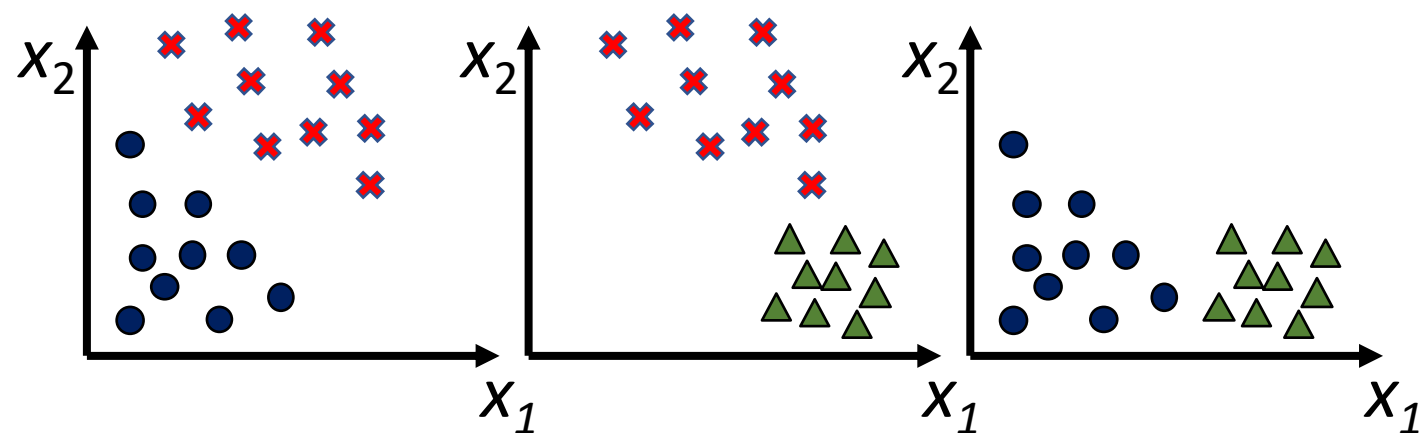
Application now to classification: One vs. One



Consider our data to have N classes...

One vs. One

- formulate $N(N-1)/2$ binary classifier models



- pick the class that receives the most positive identifications

- **Binary case:** we just need to find the optimal decision boundary that partitions these classes

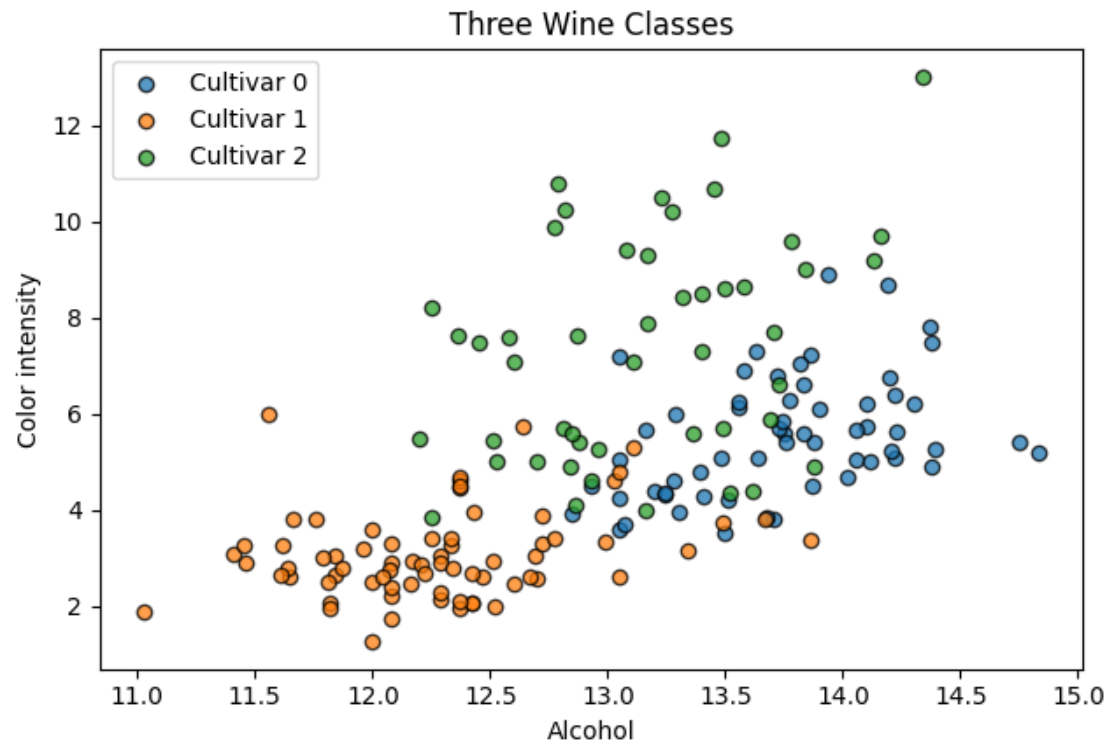
what do we do for multiple classes?

- **Multiclass case:** there are multiple strategies

Now let's put the two multiclass strategies into code

Same base classifier: Logistic Regression

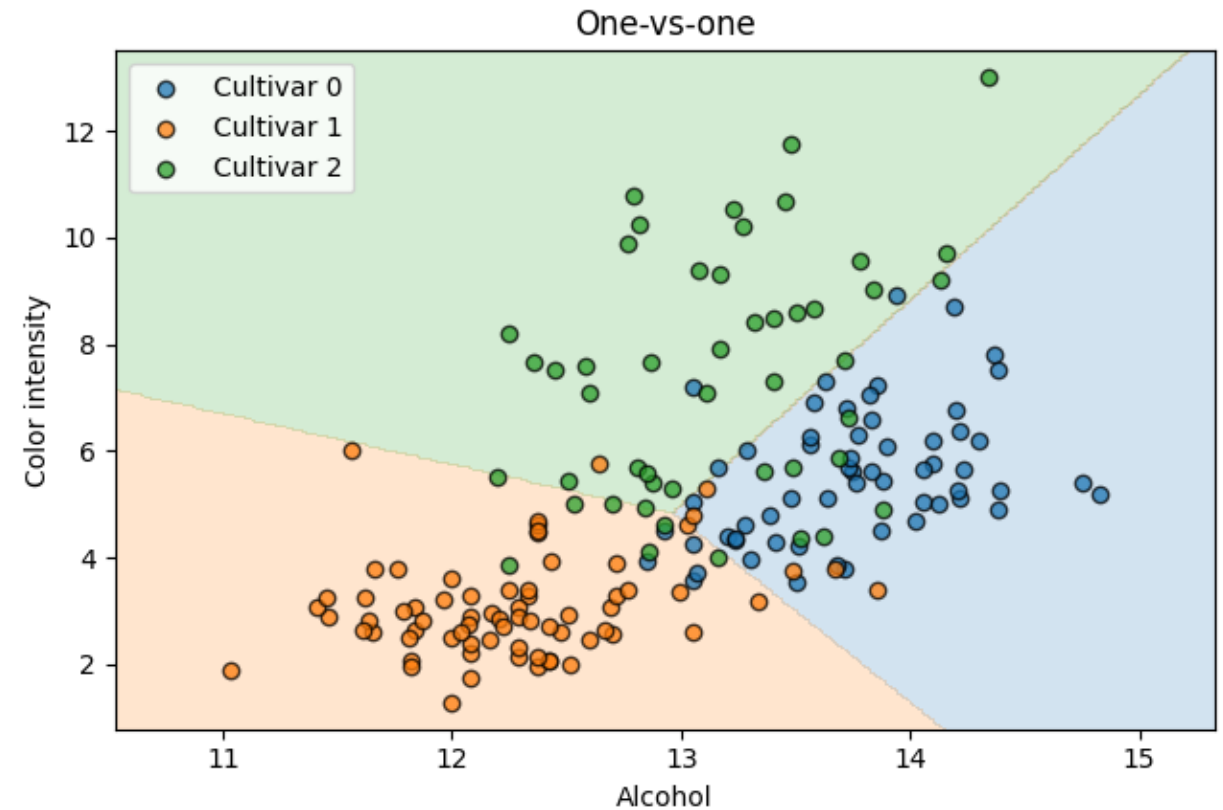
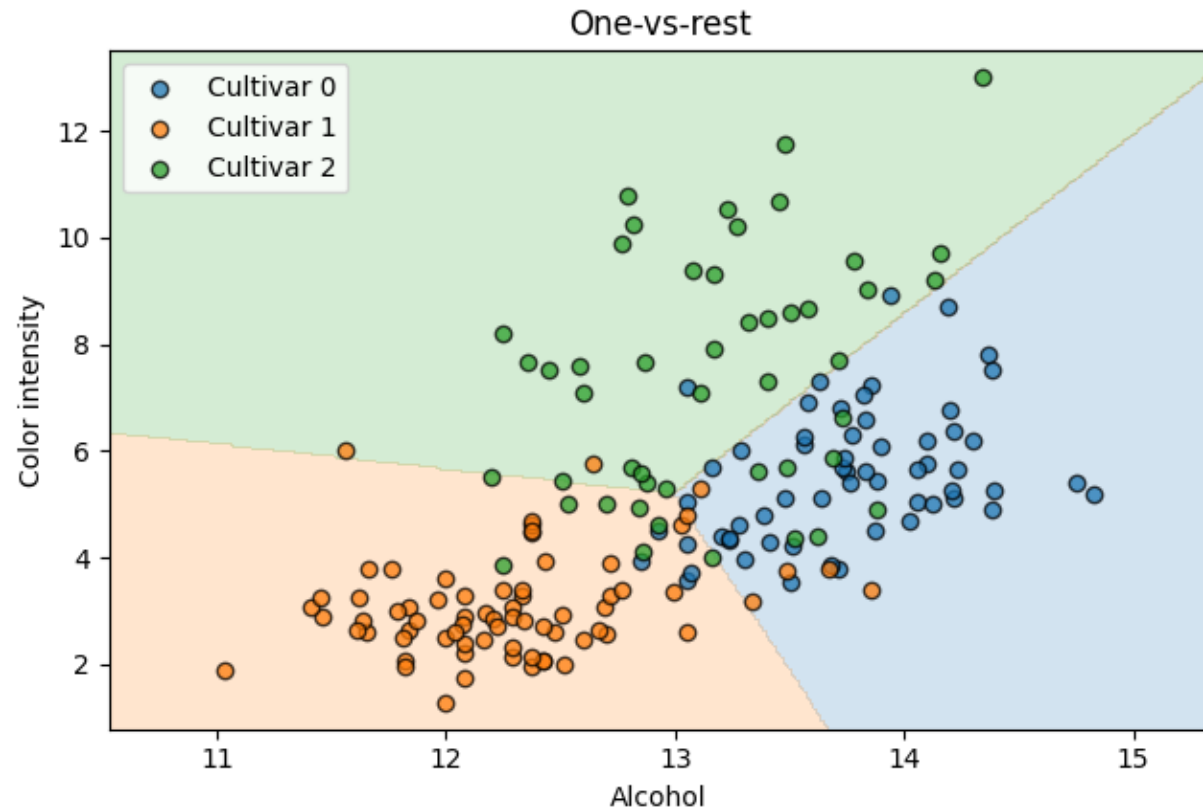
Different wrapper: One-vs-Rest vs One-vs-One



```
from sklearn.multiclass import OneVsOneClassifier, OneVsRestClassifier

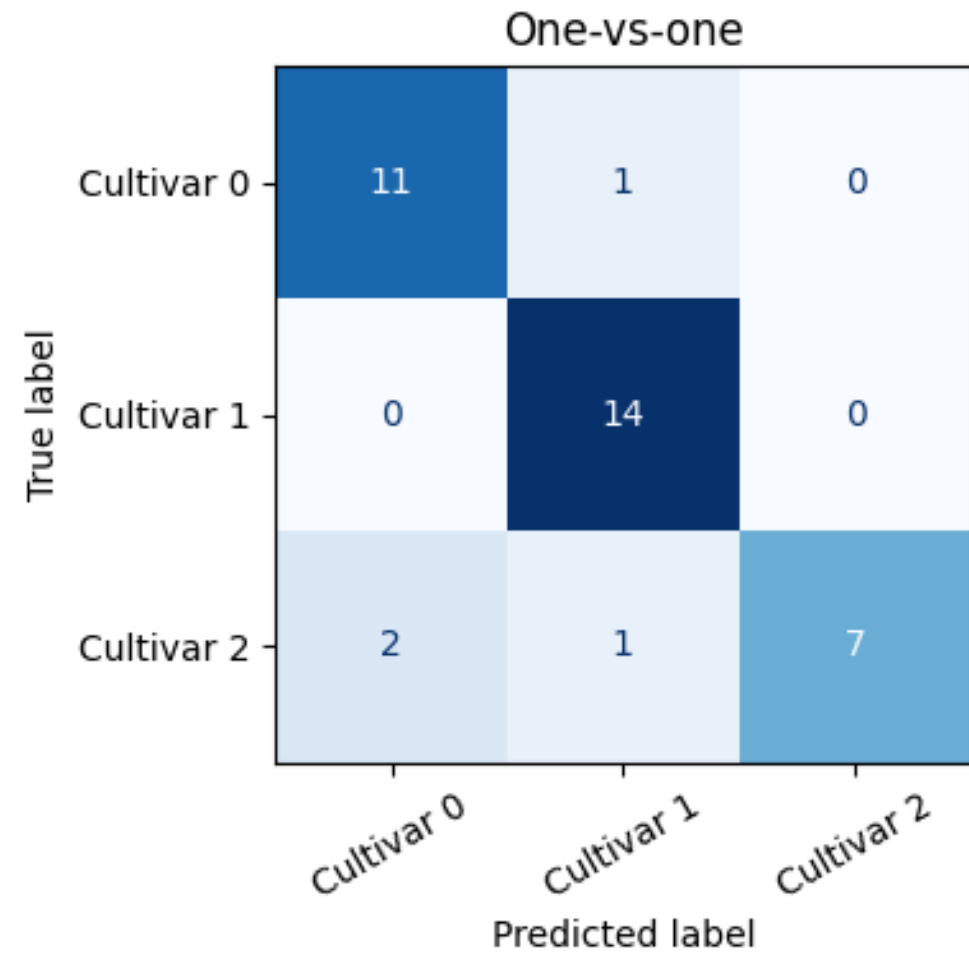
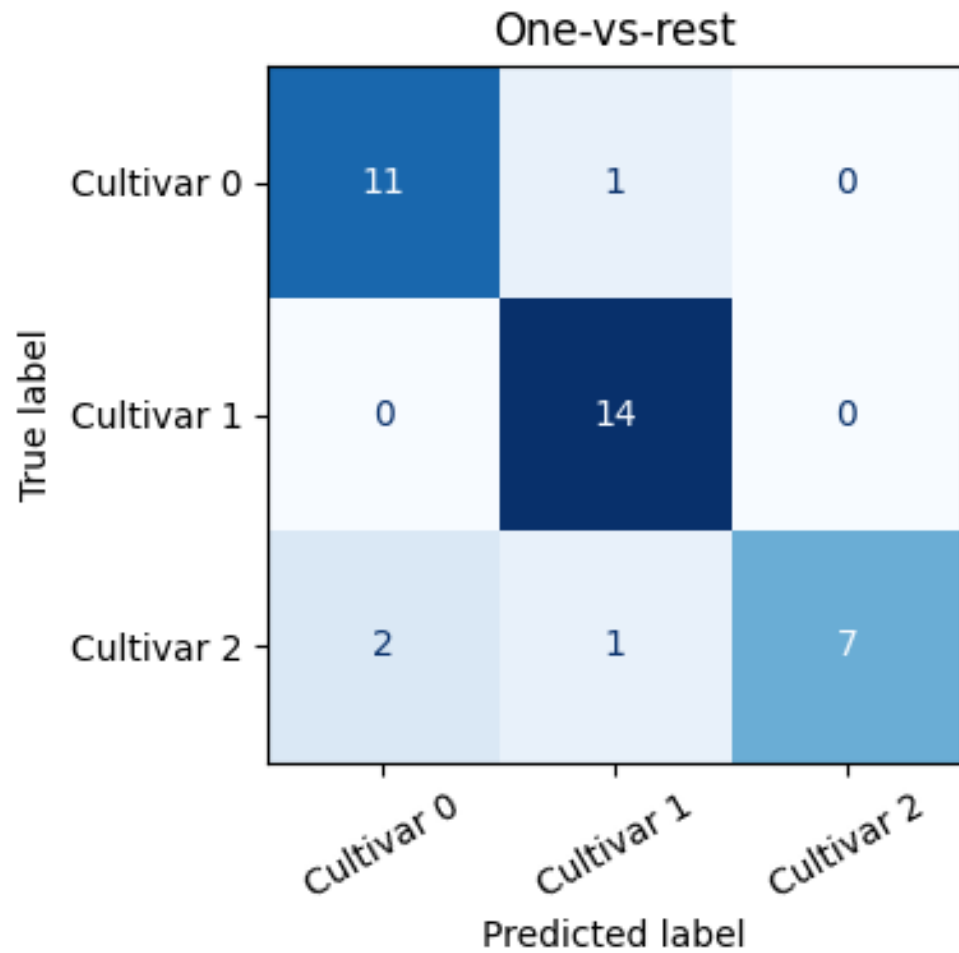
multiclass_models = {
    "One-vs-rest": Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", OneVsRestClassifier(
            LogisticRegression(max_iter=2000)
        ))
    ]),
    "One-vs-one": Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", OneVsOneClassifier(
            LogisticRegression(max_iter=2000)
        ))
    ])
}
```

Decision Boundaries: OvR vs OvO



Different multiclass strategy --> slightly different decision boundaries.

How Did They Perform?



Different multiclass strategies do not always lead to different test performance.

Quiz

Suppose we have 4 classes and use one-vs-rest (OvR) classification.

Classifier	Score
A vs rest	0.7
B vs rest	-0.4
C vs rest	1.8
D vs rest	0.2

$$\text{score} = \mathbf{w}^T \mathbf{x} + b$$

Which class will be predicted
for this sample?

The score for logistic regression is the **log-odds** (or decision value).

Pick the class with the highest score.

Quiz

Suppose we have 4 classes, and we have one-vs-one classification

Classifier	Winner
A vs B	A
A vs C	C
A vs D	A
B vs C	C
B vs D	D
C vs D	C

Count the votes. Which class is predicted?

What if there is a tie?

First count votes. If votes tie, use how strongly the pairwise classifiers supported each class.

Support Vector Machine (SVM)

“Support-vector networks”, Corinna Cortes & Vladimir Vapnik, 1995



Four Key SVM Ideas:

1. Margin
2. Support Vectors
3. Kernel
4. Regularization parameter

Input: 2D array (feature matrix)

Output: 1D array (vector of classes)

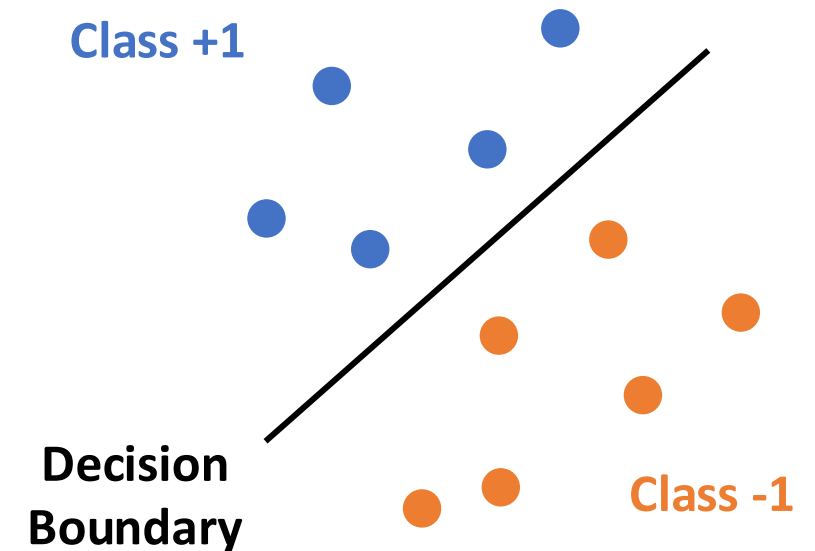
Linear Classifier: Decision Boundary

We seek a linear boundary that separates two classes.

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

- $f(\mathbf{x}) > 0$ for all $\mathbf{x}$ belong to class +1 (e.g., positive).
- $f(\mathbf{x}) < 0$ for all $\mathbf{x}$ belong to class -1 (e.g., negative).
- $f(\mathbf{x}) = 0$ for all $\mathbf{x}$ on the decision boundary.

- $\mathbf{w}$: orientation of the boundary
- b : position of the boundary



When is a point correctly classified?

We seek a linear boundary that separates two classes.

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

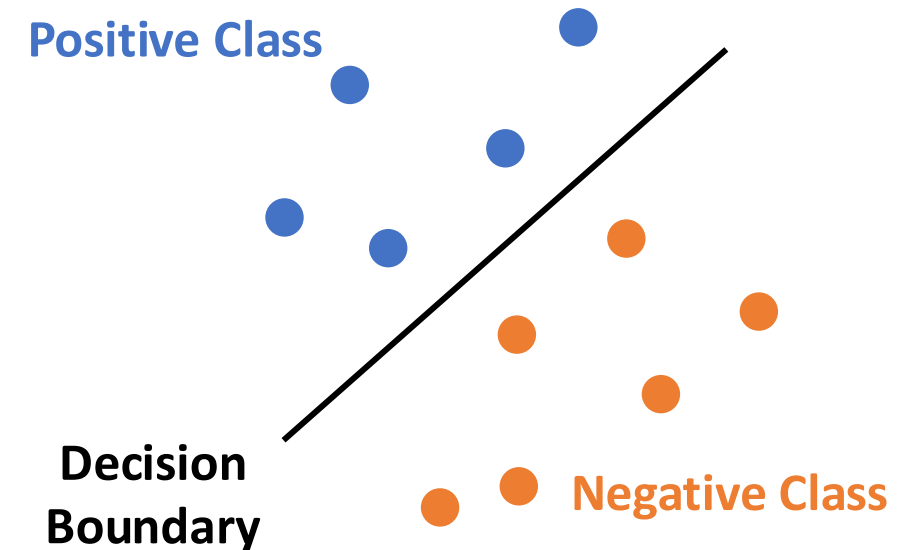
$$y \in \{-1, +1\}$$

A data point $(\mathbf{x}, y)$ is classified **correctly** if

$$y(\mathbf{w}^T \mathbf{x} + b) > 0$$

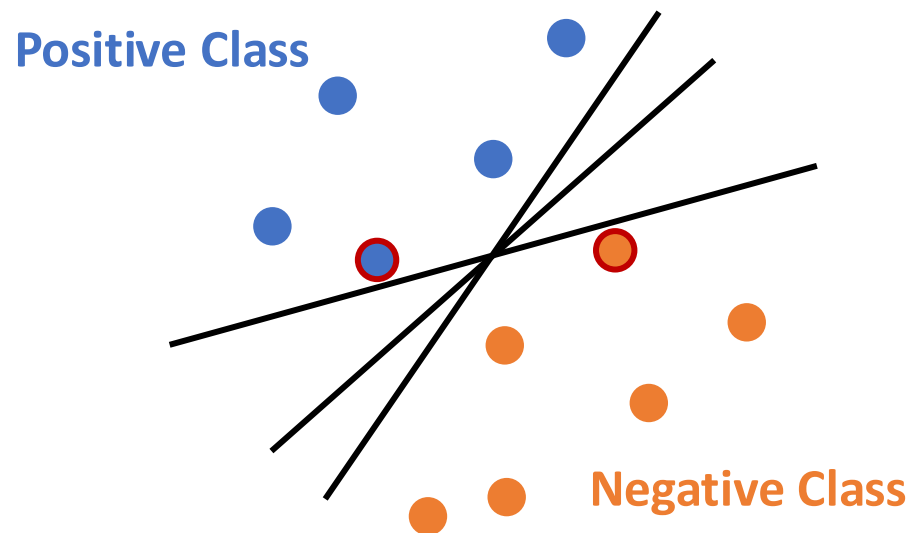
- When $y > 0$, we want?
- When $y < 0$, we want?

Both collapse into one equation.



**But there are many separating lines.
Which one should we choose?**

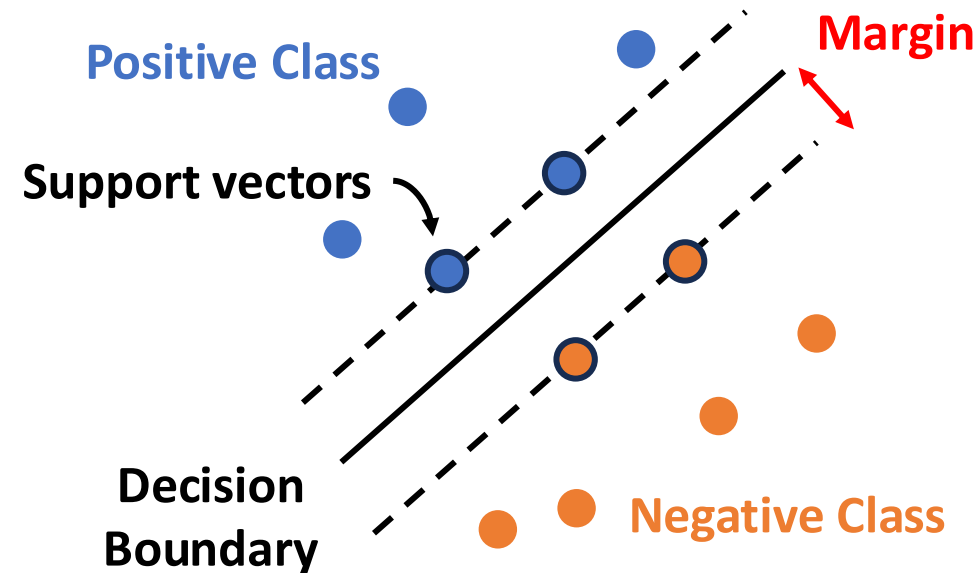
Large-Margin Classification



Many boundaries can separate the two classes.

But some boundaries pass very close to the training points.

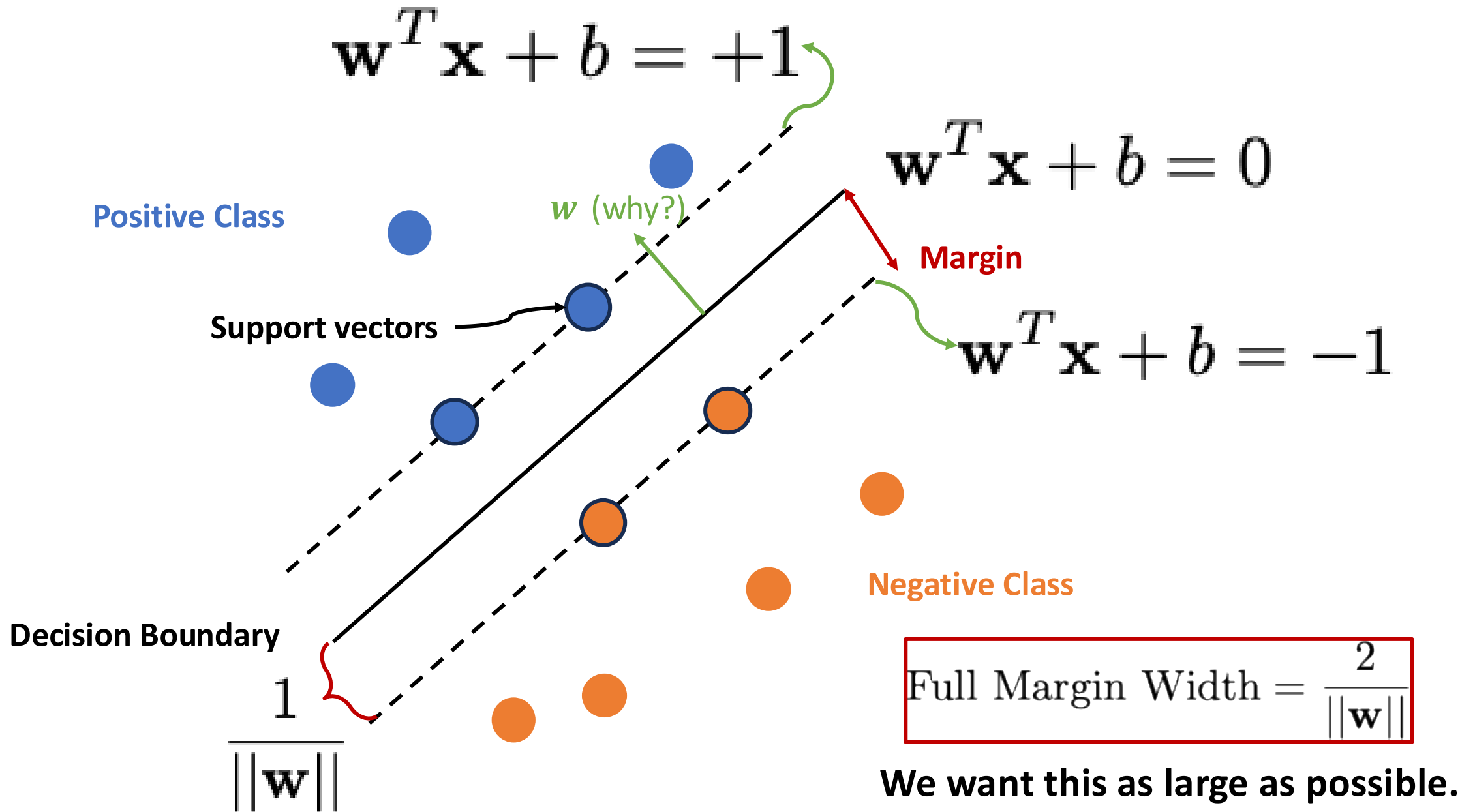
SVM chooses the boundary with the **largest margin**.



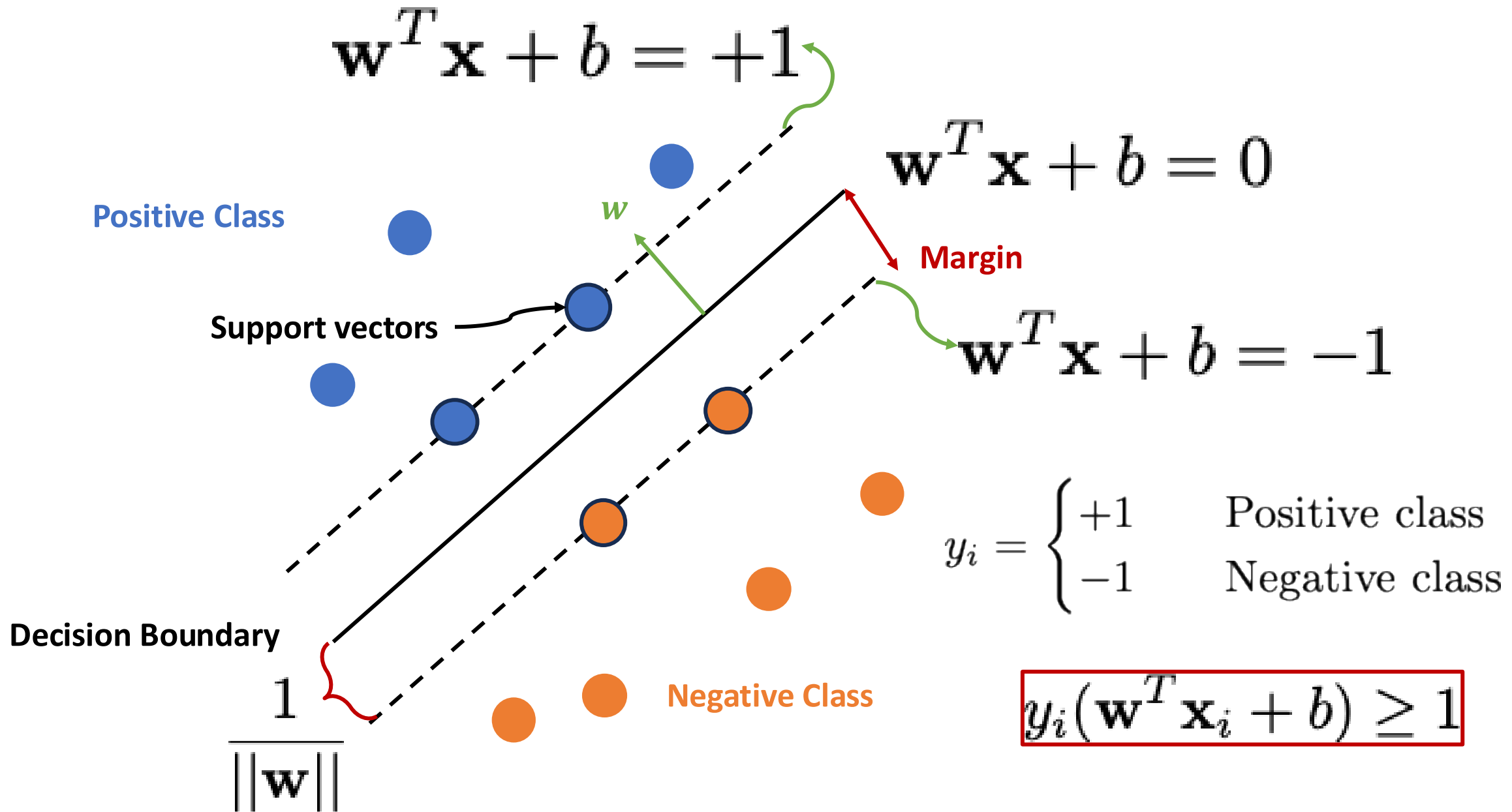
- **Support vectors** are the points that determine the position of the boundary.
- **Margin** is the distance between the decision boundary and the closest data points from either class.

Why is support "vector" a "point" here?

From Decision Boundary to Margin



From Decision Boundary to Margin



Maximum Margin as an Optimization Problem

Optimization:

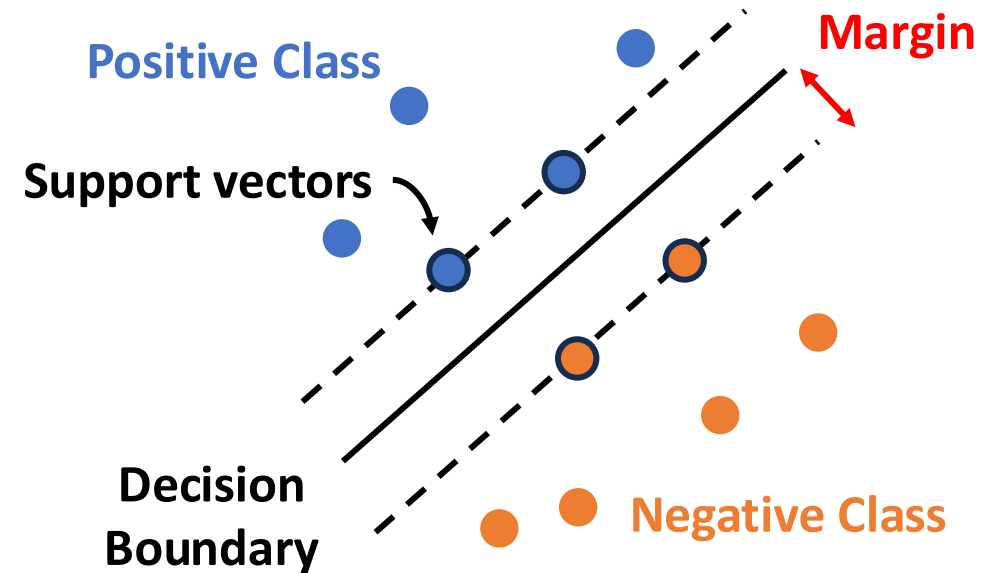
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to :

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, 2, \dots, m$$

- The **support vectors** are the active constraints that pin down the maximum-margin boundary.

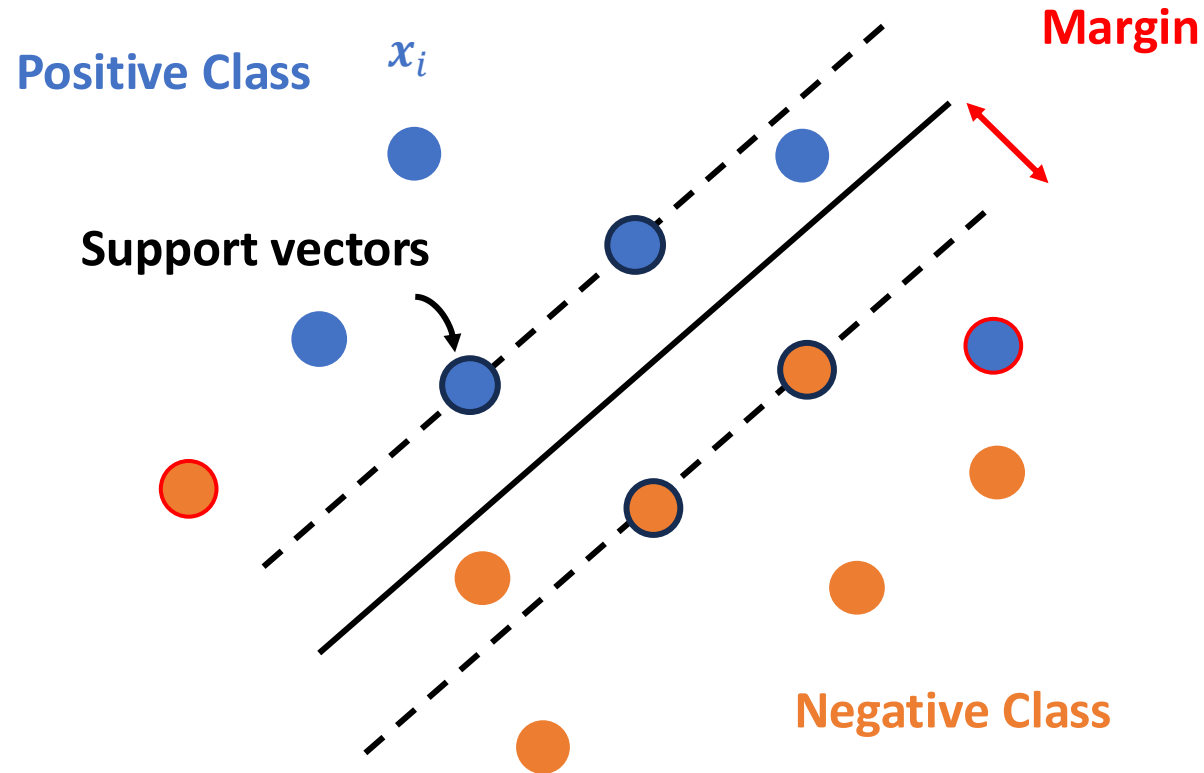
$$y_i(\mathbf{x}^T \mathbf{x}_i + b) = 1$$



What If the Classes Overlap?

Should one unusual point determine the entire boundary?

We can allow some violations to obtain a better overall boundary.



- *Real data are often noisy or overlapping.*
- *Requiring every point to be perfectly separated can give a poor boundary or no feasible hard-margin solution.*

Soft-Margin SVM

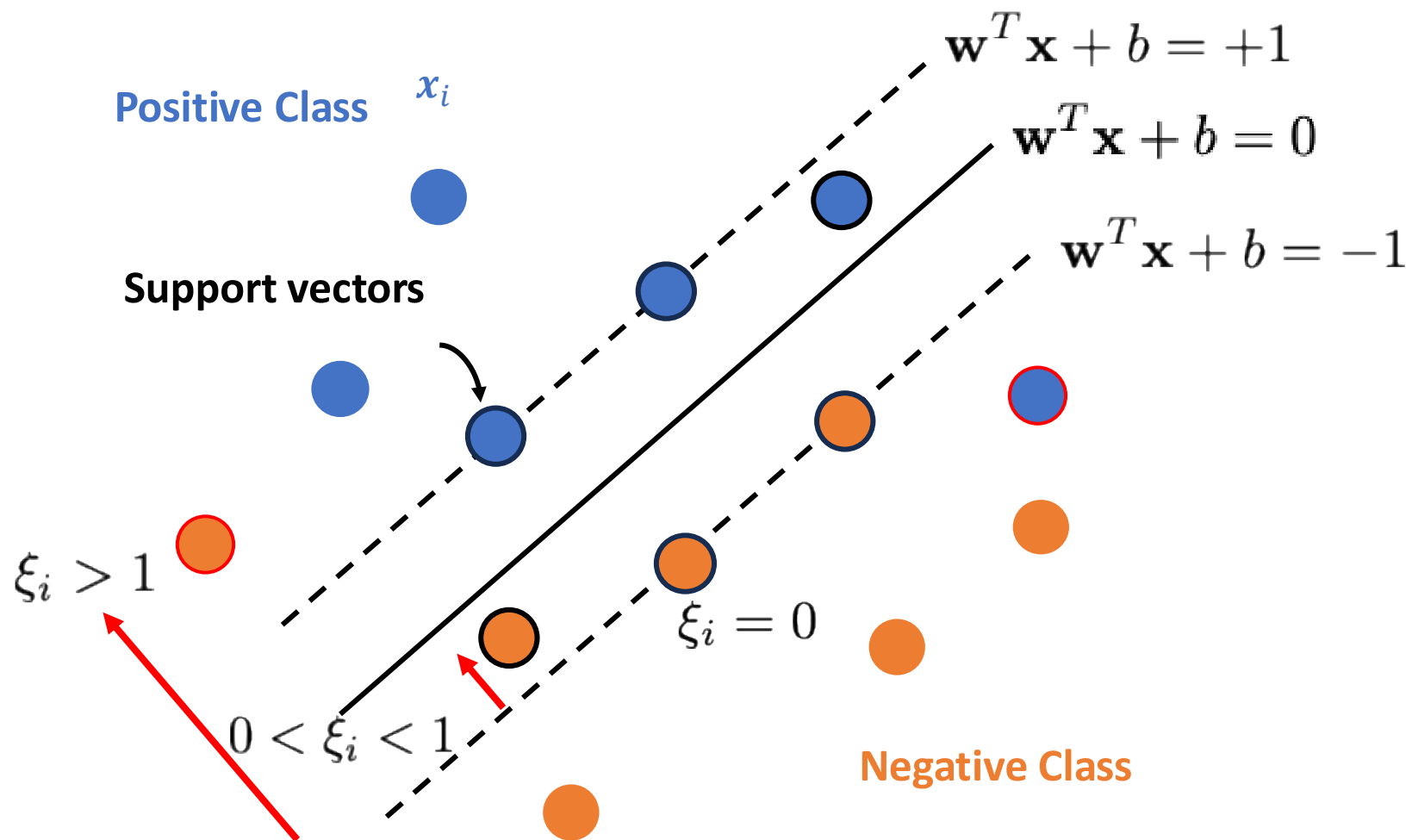
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

want a wide margin

penalize violations

subject to :

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall i$$
$$\xi_i \geq 0, \quad \forall i$$



- Smaller C : weaker penalty for violations, stronger regularization, often a wider margin.
- Larger C : the model may accept a narrower margin to reduce violations.

Soft-Margin SVM

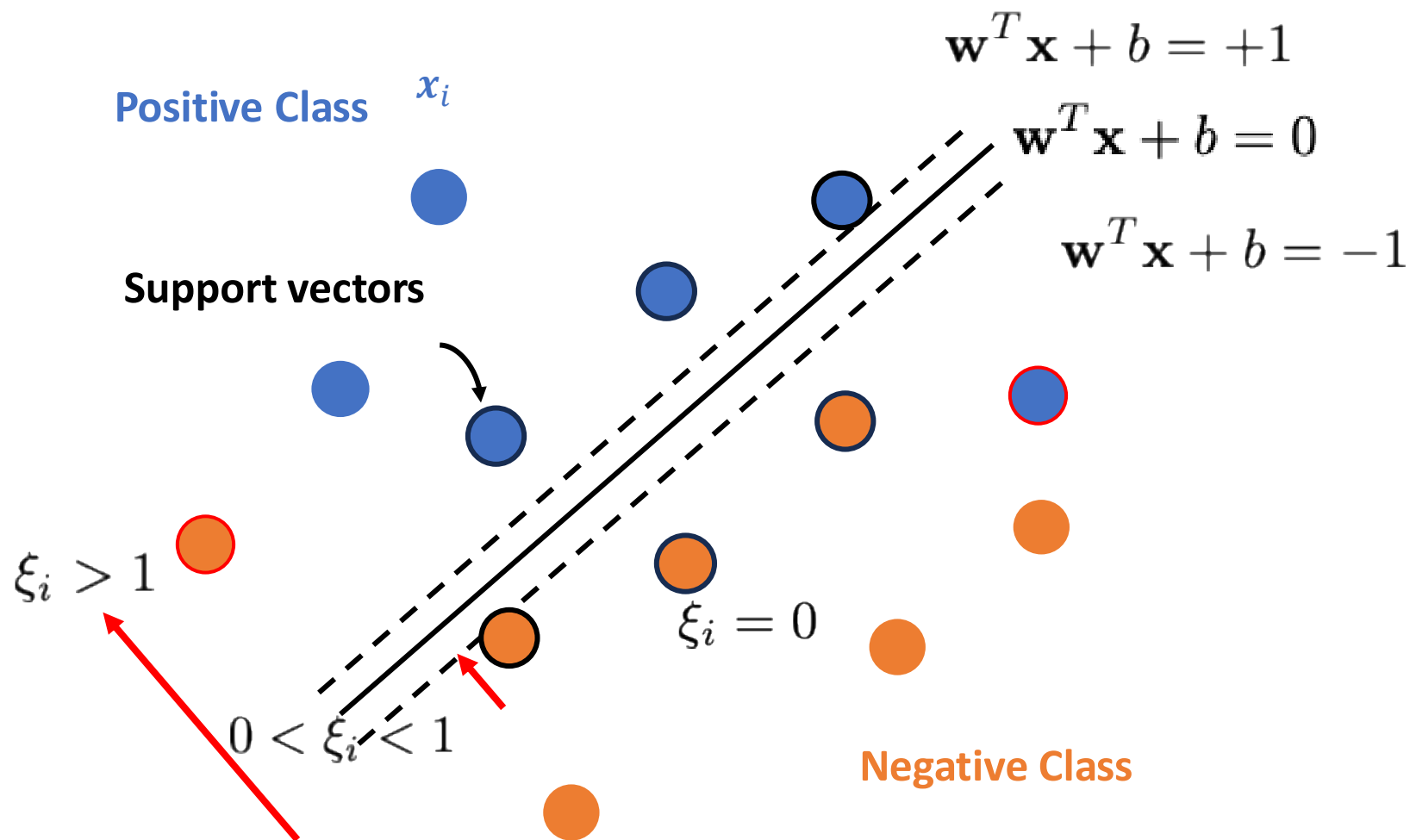
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

want a wide margin

penalize violations

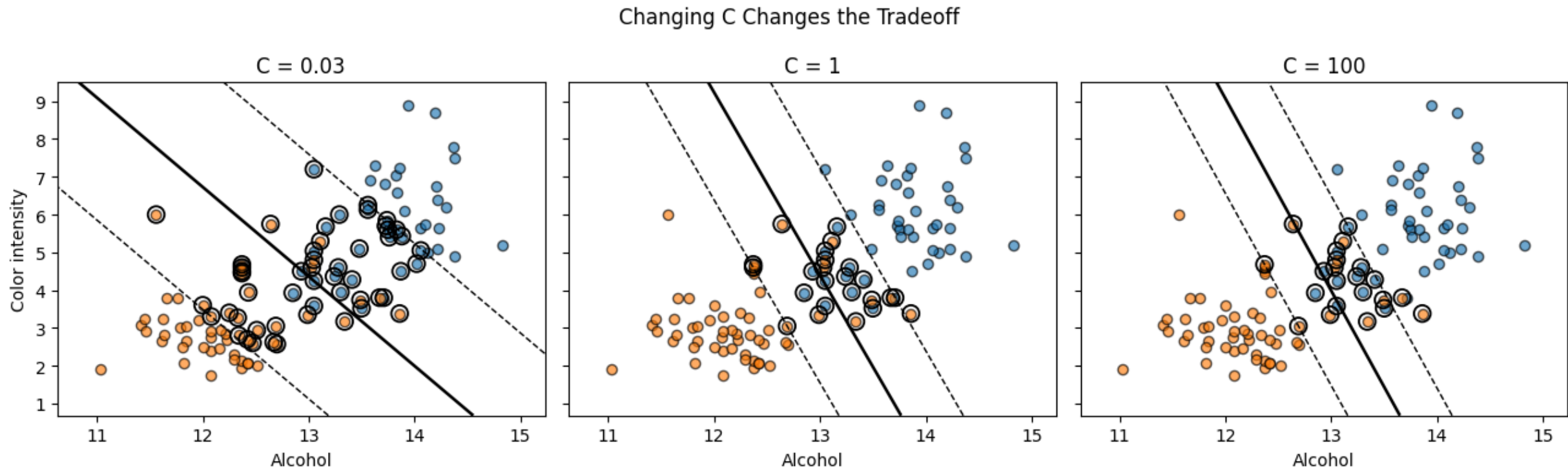
subject to :

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall i$$
$$\xi_i \geq 0, \quad \forall i$$



- Smaller C : weaker penalty for violations, stronger regularization, often a wider margin.
- Larger C : the model may accept a narrower margin to reduce violations.

Soft-Margin SVM

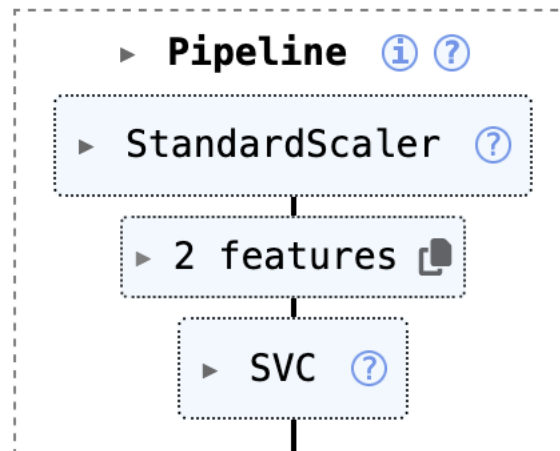


- Smaller C : weaker penalty for violations, stronger regularization, often a wider margin.
- Larger C : the model may accept a narrower margin to reduce violations.

Example

```
class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale',  
coef0=0.0, shrinking=True, probability='deprecated', tol=0.001,  
cache_size=200, class_weight=None, verbose=False, max_iter=-1,  
decision_function_shape='ovr', break_ties=False, random_state=None)
```

```
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC  
  
svm_linear = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm", SVC(kernel="linear", C=1))  
)  
  
svm_linear.fit(X_train, y_train)
```



Multiclass Linear SVM Example

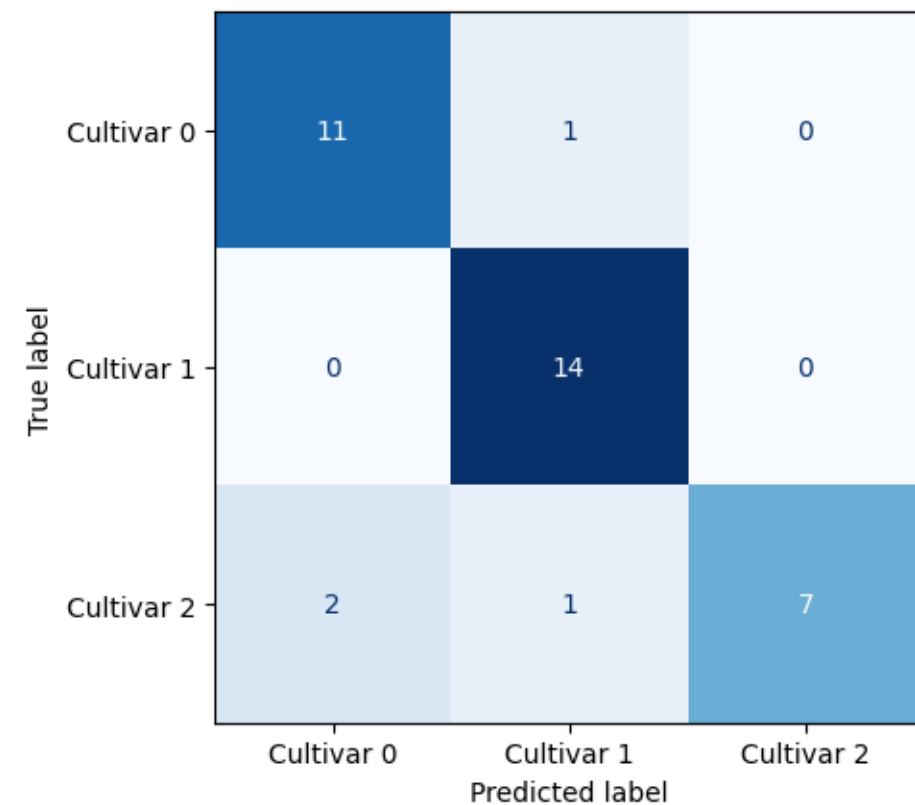
```
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay

y_pred = svm_linear.predict(X_test)

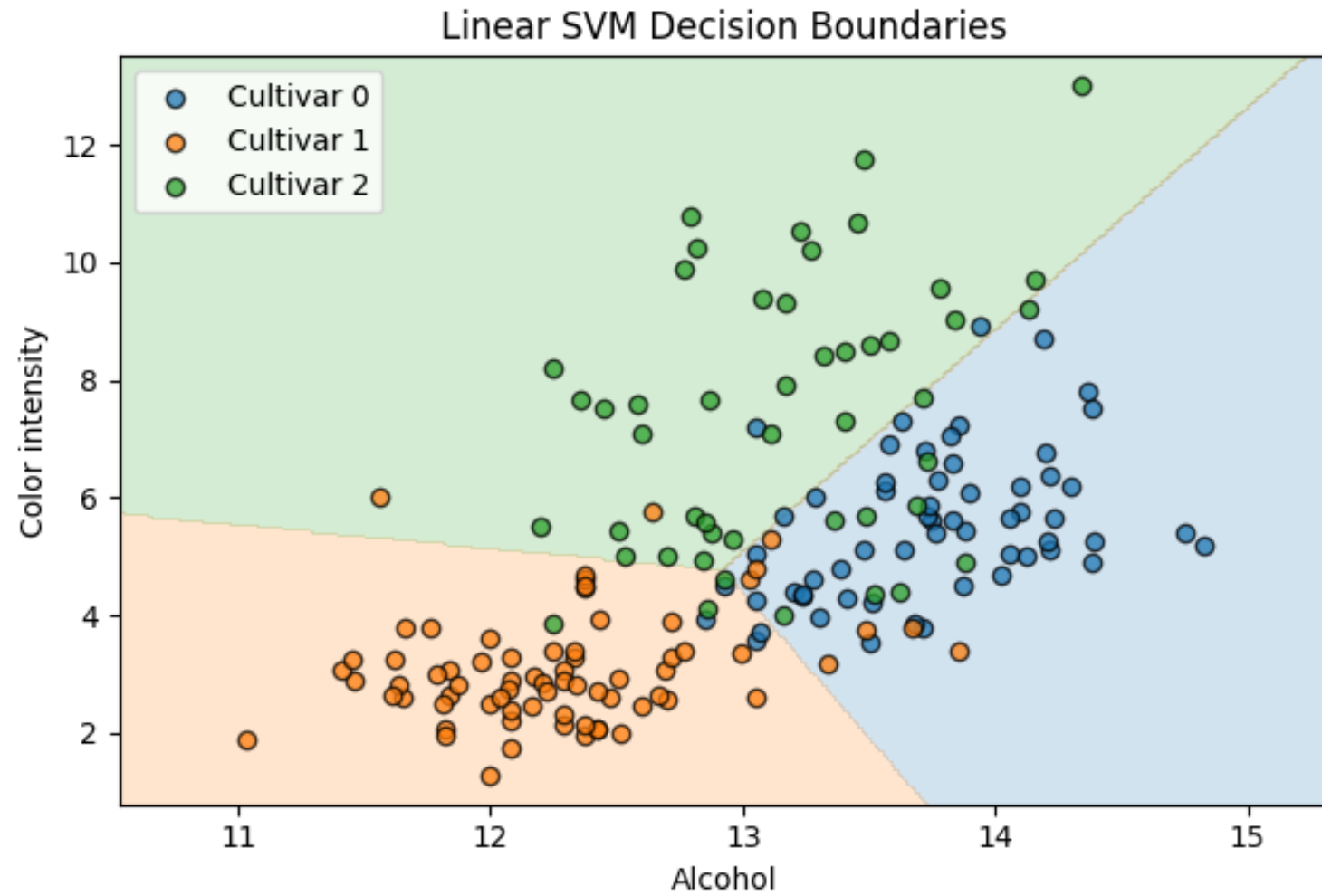
print("Accuracy:", accuracy_score(y_test, y_pred))

ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
                                       display_labels=["Cultivar 0", "Cultivar 1", "Cultivar 2"],
                                       cmap="Blues",
                                       colorbar=False)
plt.show()
```

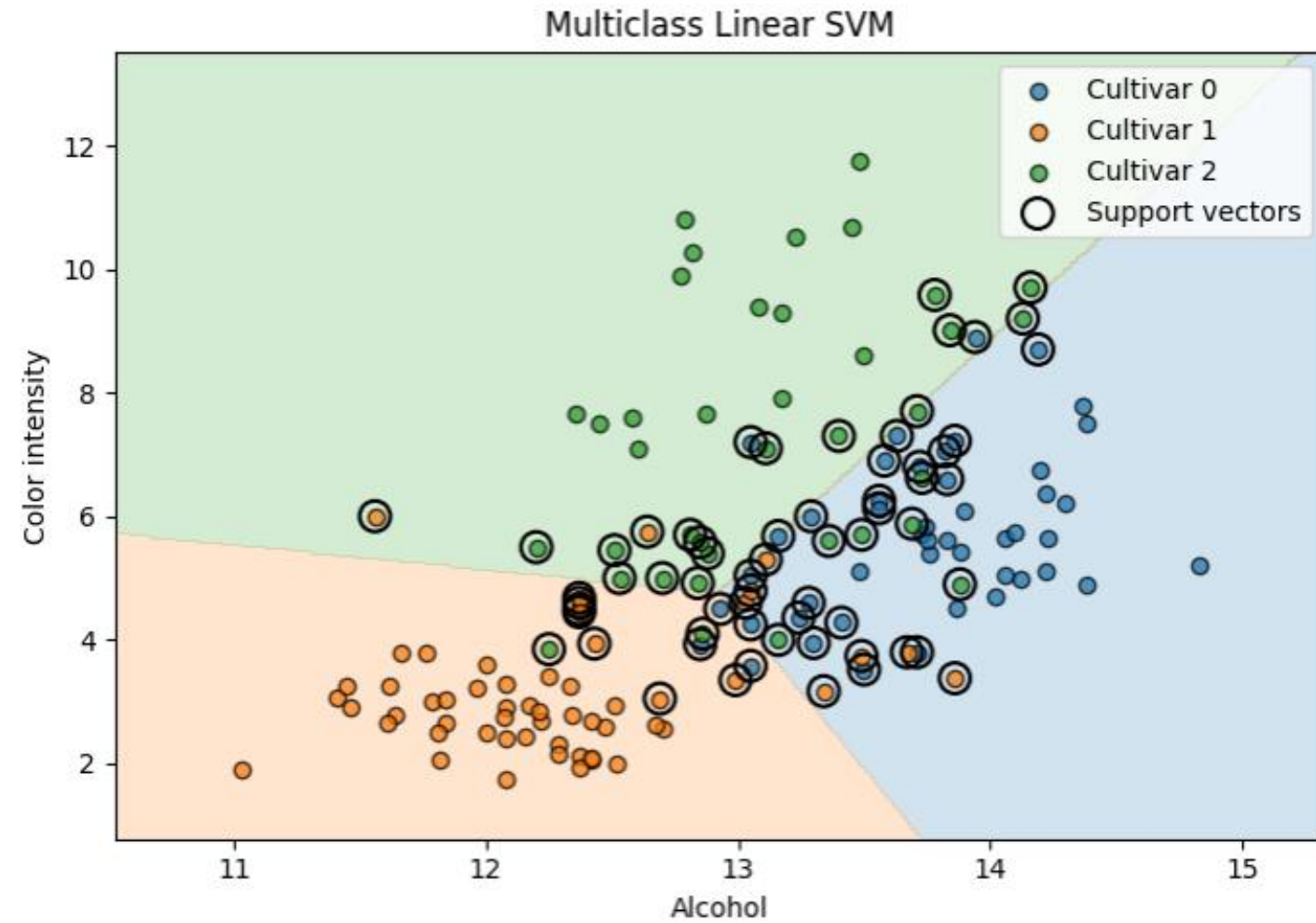
Accuracy: 0.8888888888888888



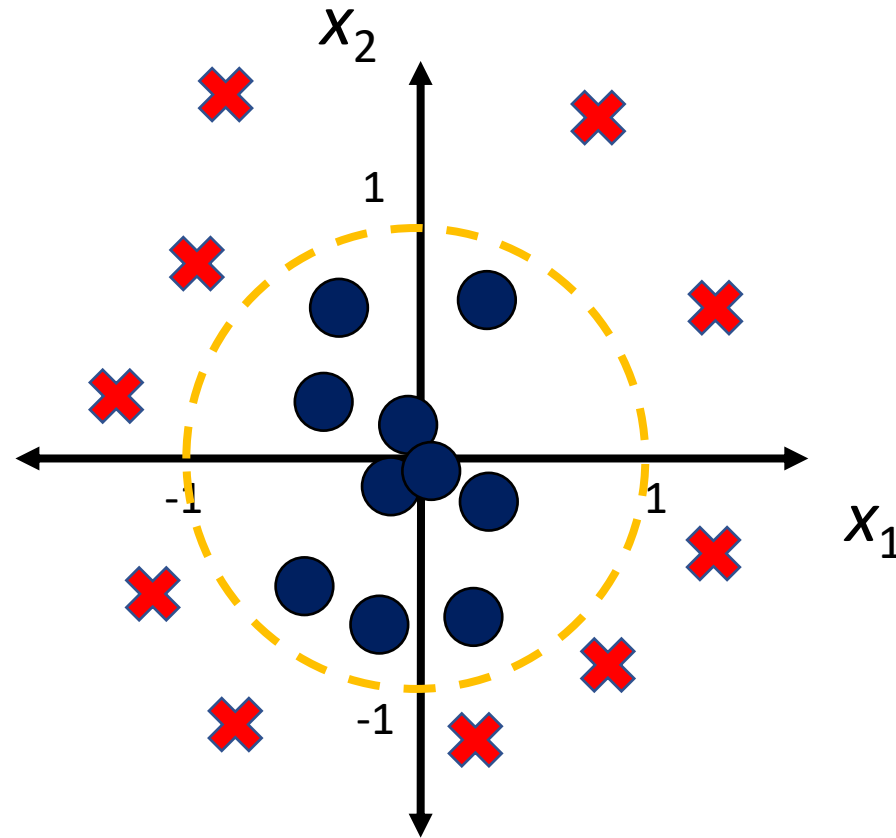
Multiclass Linear SVM Example



Multiclass Linear SVM Example



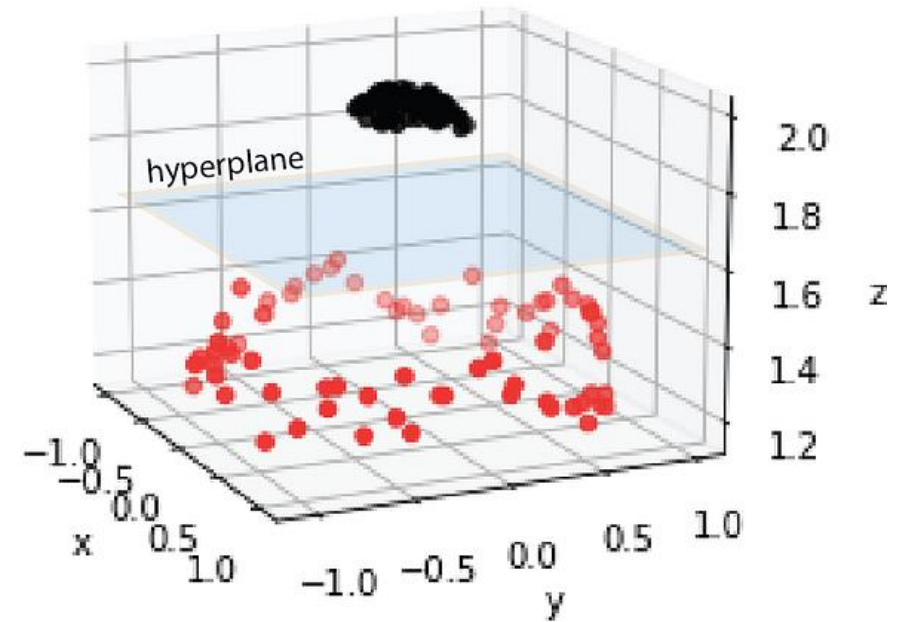
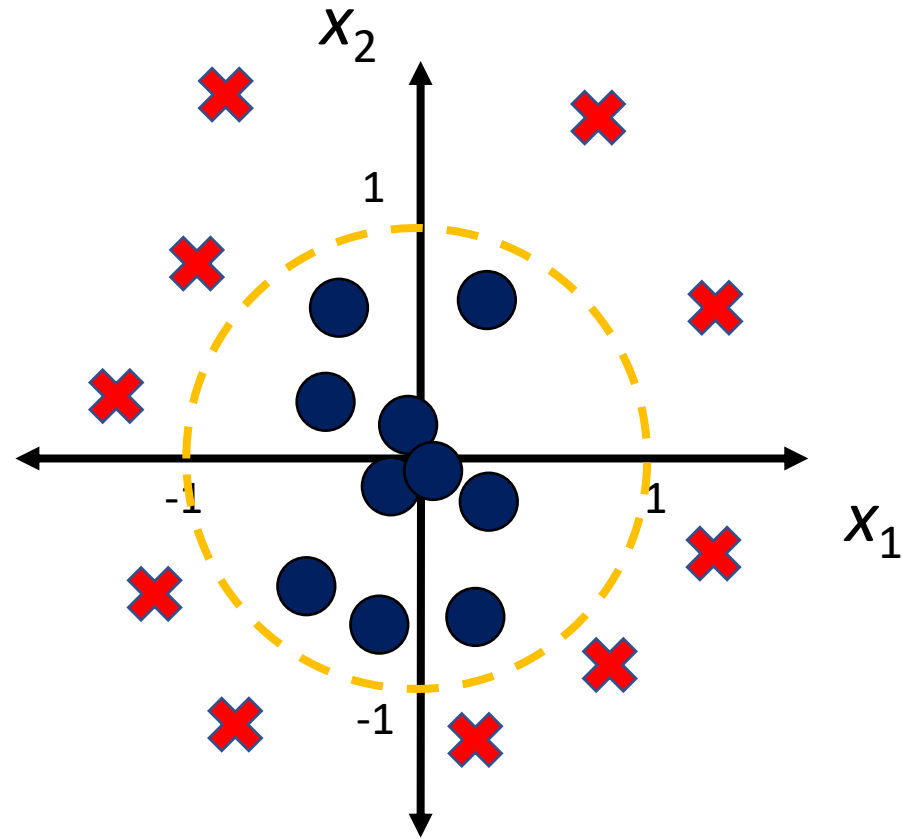
What if a line cannot separate the data?



$$f(x_1, x_2) = \frac{1}{1 + e^{-z}}, \quad z = x_1^2 + x_2^2$$

Map the features to a new space where a linear separator may become possible.

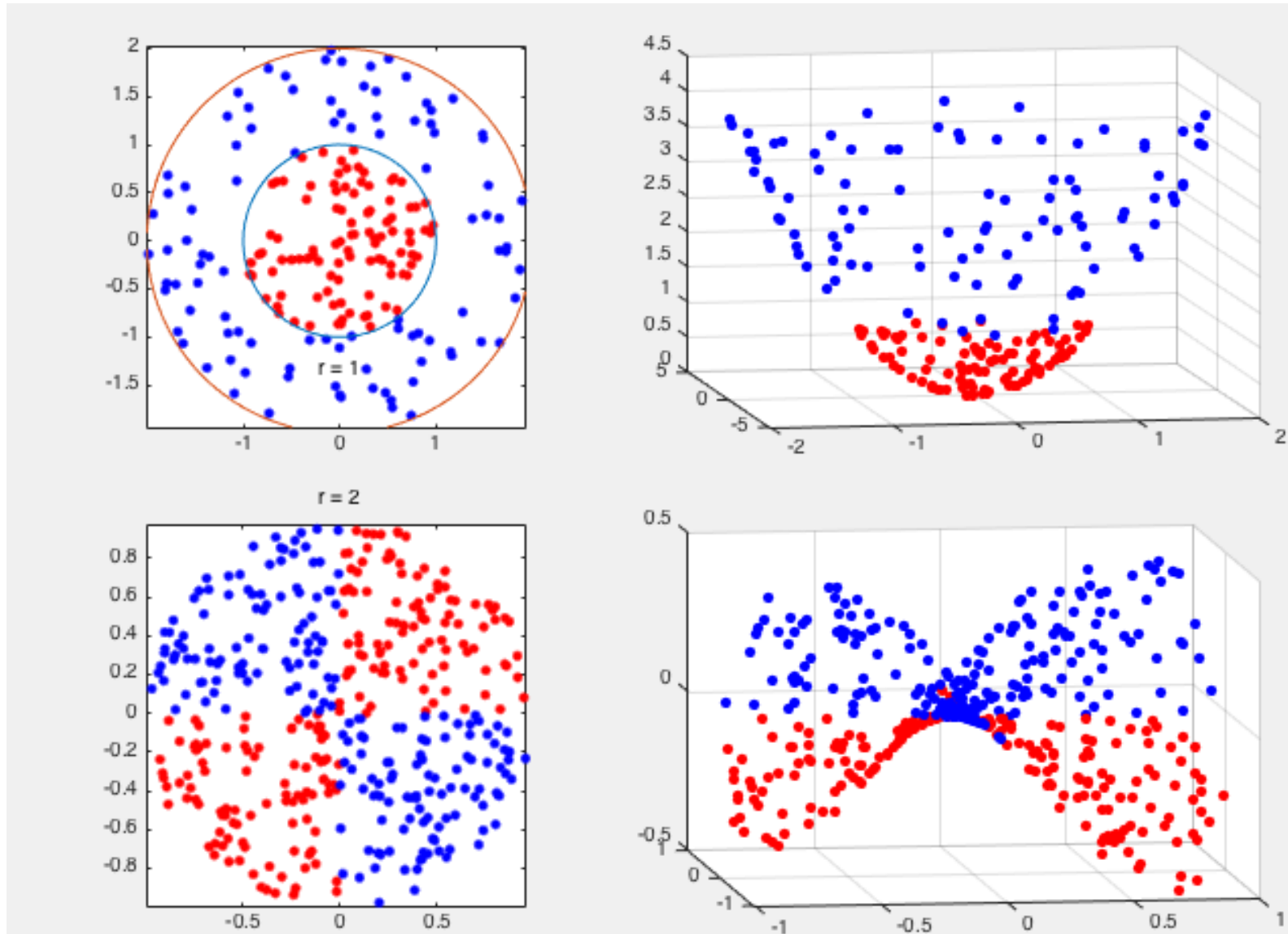
What if a line cannot separate the data?



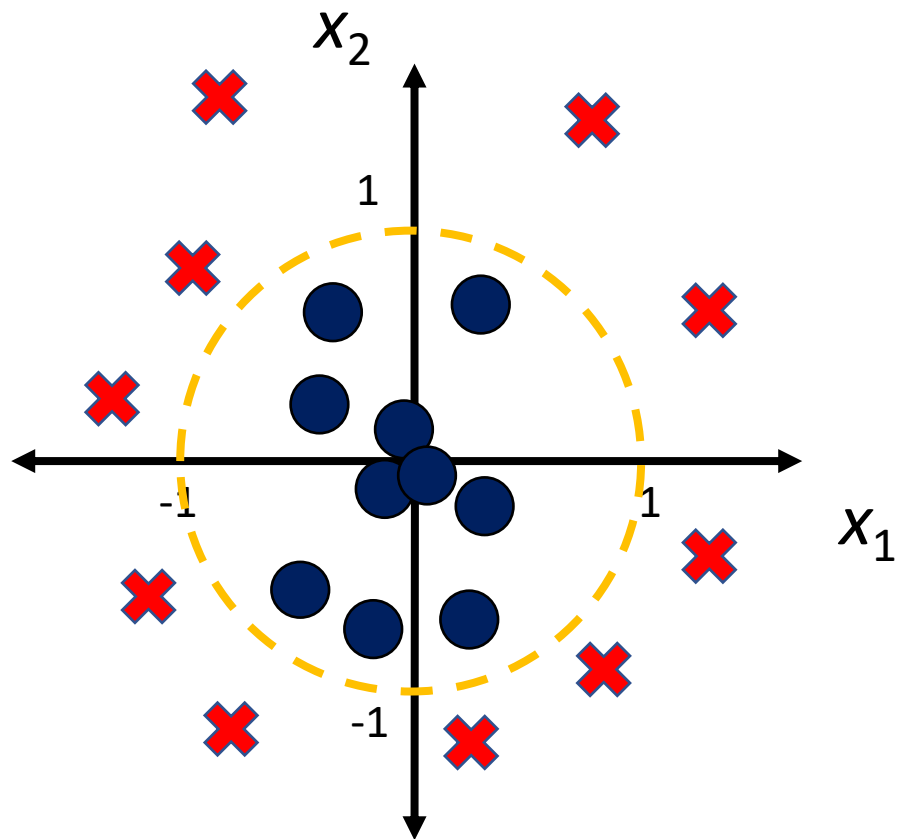
$$f(x_1, x_2) = \frac{1}{1 + e^{-z}}, \quad z = x_1^2 + x_2^2$$

Nonlinear boundary in the original space $\rightarrow$ linear boundary in the transformed space.

What if a line cannot separate the data?



Feature Mapping



$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

$$\phi(\mathbf{x}) = x_1^2 + x_2^2$$

Decision boundary

$$\mathbf{w}^T \phi(\mathbf{x}) + b = 0$$

SVM is still finding a **linear hyperplane**, just in the transformed feature space.

But do we need to calculate every $\phi(x)$?

What if $\phi(x)$ has thousands or infinitely many features?

The Kernel Trick

SVM does not actually need the transformed coordinates.
It only needs dot products between transformed points.

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

SVM $f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b$

After solving the optimization problem

$$\mathbf{w} = \sum_i \alpha_i y_i \phi(\mathbf{x}_i)$$

α_i 's are learned coefficients.

$$f(\mathbf{x}) = \left(\sum_i \alpha_i y_i \phi(\mathbf{x}_i) \right)^T \phi(\mathbf{x}) + b$$

$$f(\mathbf{x}) = \sum_i \alpha_i y_i \underbrace{\phi(\mathbf{x}_i)^T \phi(\mathbf{x})}_{\text{dot product}} + b$$

**Two decision boundaries classify every training point correctly.
Which one will SVM prefer?**

- A. The one closest to the most data points
- B. The one with the largest margin
- C. Either one, because training accuracy is the same
- D. The one with the steepest slope

**One unusual point violates the margin.
What happens if we increase C ?**

- A. We allow violations more easily
- B. We penalize violations more strongly
- C. We automatically create a nonlinear boundary
- D. We remove the support vectors

Why can SVM use a kernel without explicitly calculating the transformed features

- A. SVM does not use the training data after transformation
- B. The kernel directly predicts the class
- C. SVM only needs dot products between transformed points
- D. The kernel removes the optimization problem

Interactive Python Time

`13_support_vector_machine.ipynb`